Library Manager

A Spring Boot application for managing Authors and Books

Project documentation: approach, entity design, implementation, testing and challenges

Name of student	Roll number
Roshan Avadhesh Singh	**2024eb03006**

GitHub repository

https://github.com/roshanasingh4/library-manager-springboot

Item	Detail
Entities	Author and Book (one author, many books)
Operations	Create, Read (including a custom inner join query) and Update
Technology	Java 21, Spring Boot 3.3.5, Spring MVC, Spring Data JPA (Hibernate 6), H2, JSP with JSTL, JUnit 5, Mockito
Tests	25 unit tests (repository and service layers), all passing

Contents

1. Approach and architecture
2. Entity relationship design
3. Populating the database
4. Create operation
5. Read operation and custom inner join
6. Update operation
7. Exception handling
8. User interface
9. Testing
10. Challenges faced and solutions
11. Running the project

1. Approach and architecture

The application manages a small library catalogue made up of two related entities, **Author** and **Book**. I chose this pair because the relationship is intuitive (an author writes many books, and each book has exactly one author) and because both entities have a natural unique field, the author's e-mail address and the book's ISBN, which makes it possible to demonstrate data integrity handling realistically.

The project follows the standard layered architecture of a Spring Boot web application, with each layer having a single responsibility:

Layer	Classes	Responsibility
Entity	Author, Book	JPA-mapped domain model, relationships and column constraints
Repository	AuthorRepository, BookRepository	Spring Data JPA interfaces with derived queries and custom JPQL inner joins
Service	AuthorService, BookService	Business rules, uniqueness checks, transactions, translation of integrity violations
Controller	HomeController, AuthorController, BookController	Handle HTTP requests, validate form input, bind data to views, redirect after POST
View	JSP pages with JSTL / EL, app.css	Forms and tables rendered on the server and styled with CSS
Support	BookForm, BookAuthorView, AuthorSummary, exceptions, DataSeeder	Form-backing and projection DTOs, error handling and sample data

A request therefore flows from the browser to a controller, which calls a service, which uses a repository to read or write the H2 database through Hibernate. The controller places the result in the Spring `Model`, and the JSP view reads it with Expression Language. After a successful create or update the controller issues a redirect (the Post/Redirect/Get pattern) so that refreshing the page never resubmits the form.

```
com.roshan.library
  LibraryManagerApplication.java
  entity/       Author.java, Book.java
  repository/   AuthorRepository.java, BookRepository.java
  service/      AuthorService.java, BookService.java
  controller/   HomeController.java, AuthorController.java, BookController.java
  dto/          BookForm.java, BookAuthorView.java, AuthorSummary.java
  exception/    DuplicateResourceException.java, ResourceNotFoundException.java,
                GlobalExceptionHandler.java
  config/       DataSeeder.java
resources/META-INF/resources/WEB-INF/jsp/   home.jsp, authors/*.jsp, books/*.jsp, common/*
resources/static/css/app.css
```

Figure 1. Package structure of the project

2. Entity relationship design

The two tables are linked by a foreign key `books.author_id` that references `authors.id`. On the Java side this is a bidirectional association: `Author` owns a `@OneToMany` list of books, and `Book` has a `@ManyToOne` reference to its author. The book side is the owning side of the relationship (it holds the join column), and `mappedBy = "author"` tells JPA not to create a separate join table.

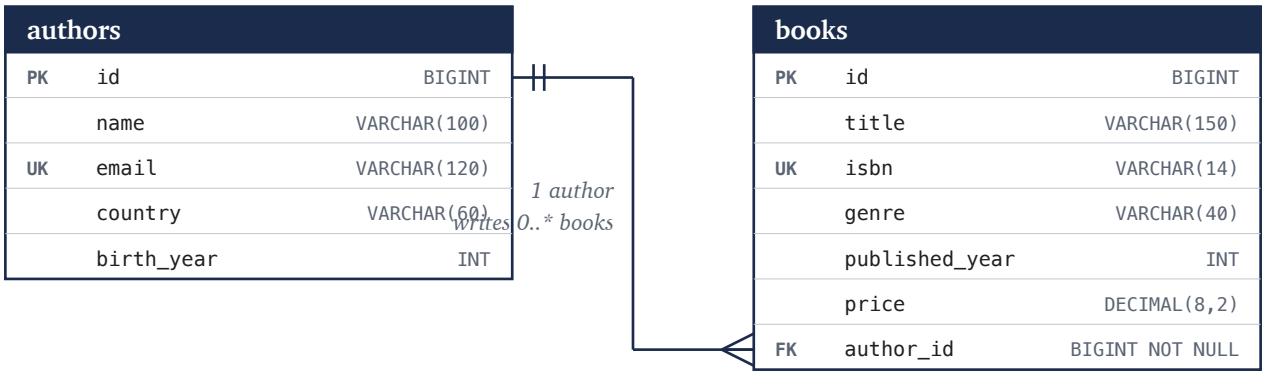


Figure 2. Entity relationship diagram (PK primary key, UK unique key, FK foreign key)

Design decision	Reason
Unique constraint on `authors.email` and `books.isbn`	Real-world identifiers must not repeat; the constraints also give the database a final line of defence against duplicates.
`author_id` is `NOT NULL` (`optional = false`)	A book cannot exist without an author, which keeps the inner join complete.
`FetchType.LAZY` on both sides	Lists are loaded only when needed; the edit page uses `JOIN FETCH` to load a book with its author in one query.
`BigDecimal` price with `precision = 8, scale = 2`	Avoids floating-point rounding errors for money.
Bean Validation annotations on the entities	`@NotBlank`, `@Email`, `@Pattern`, `@Min`/`@Max` and `@DecimalMin` reject bad input before it reaches the database.

```
@Entity
@Table(name = "authors",
       uniqueConstraints = @UniqueConstraint(name = "uk_author_email", columnNames = "email"))
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(max = 100) @Column(nullable = false, length = 100)
    private String name;

    @NotBlank(message = "E-mail is required")
    @Email(message = "Enter a valid e-mail address")
    @Column(nullable = false, length = 120)
    private String email;

    @NotBlank @Column(nullable = false, length = 60)
    private String country;

    @NotNull @Min(1500) @Max(2015) @Column(name = "birth_year", nullable = false)
    private Integer birthYear;

    @OneToMany(mappedBy = "author", fetch = FetchType.LAZY)
    @OrderBy("title ASC")
    private List<Book> books = new ArrayList<>();
    // constructors, getters and setters
}
```

Figure 3. Author entity (abridged)

```
@Entity
@Table(name = "books",
       uniqueConstraints = @UniqueConstraint(name = "uk_book_isbn", columnNames = "isbn"))
public class Book {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(max = 150) @Column(nullable = false, length = 150)
    private String title;

    @NotBlank
    @Pattern(regexp = "^97[89]-\\d{10}$", message = "ISBN must look like 978-0123456789")
    @Column(nullable = false, length = 14)
    private String isbn;

    @NotBlank @Column(nullable = false, length = 40)
    private String genre;

    @NotNull @Min(1450) @Max(2030) @Column(name = "published_year", nullable = false)
    private Integer publishedYear;

    @NotNull @DecimalMin(value = "0.0", inclusive = false) @Digits(integer = 6, fraction = 2)
    @Column(nullable = false, precision = 8, scale = 2)
    private BigDecimal price;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "author_id", nullable = false,
                foreignKey = @ForeignKey(name = "fk_book_author"))
    private Author author;
    // constructors, getters and setters
}
```

Figure 4. Book entity (abridged)

3. Populating the database

Hibernate generates both tables from the entity classes at start-up (`spring.jpa.hibernate.ddl-auto=create-drop`) in an in-memory H2 database. A `CommandLineRunner` named `DataSeeder` then inserts **10 authors and 10 books**. One author, Amitav Ghosh, is deliberately left without a book so that the difference between the inner join (which excludes him) and the full author list (which includes him) is visible in the application. The seeder is excluded from the test profile so that tests start from an empty database.

```
@Component
@Profile("!test")
public class DataSeeder implements CommandLineRunner {

    @Override
    @Transactional
    public void run(String... args) {
        if (authorRepository.count() > 0) return;

        List<Author> authors = authorRepository.saveAll(List.of(
            new Author("R. K. Narayan", "rk.narayan@library.org", "India", 1906),
            new Author("Ruskin Bond", "ruskin.bond@library.org", "India", 1934),
            ...                                          // 10 authors in total
            new Author("Amitav Ghosh", "amitav.ghosh@library.org", "India", 1956)));

        bookRepository.saveAll(List.of(
            book("Malgudi Days", "978-0143039655", "Short Stories", 1943, "299.00", narayan),
            book("The Guide", "978-0143039648", "Fiction", 1958, "250.00", narayan),
            ...                                          // 10 books in total
            book("Beloved", "978-1400033416", "Fiction", 1987, "375.00", morrison)));
    }
}
```

Figure 5. DataSeeder inserts 10 rows into each table

```
spring.datasource.url=jdbc:h2:mem:librarydb;DB_CLOSE_DELAY=-1
spring.jpa.hibernate.ddl-auto=create-drop
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
spring.h2.console.enabled=true
```

Figure 6. Key settings in application.properties

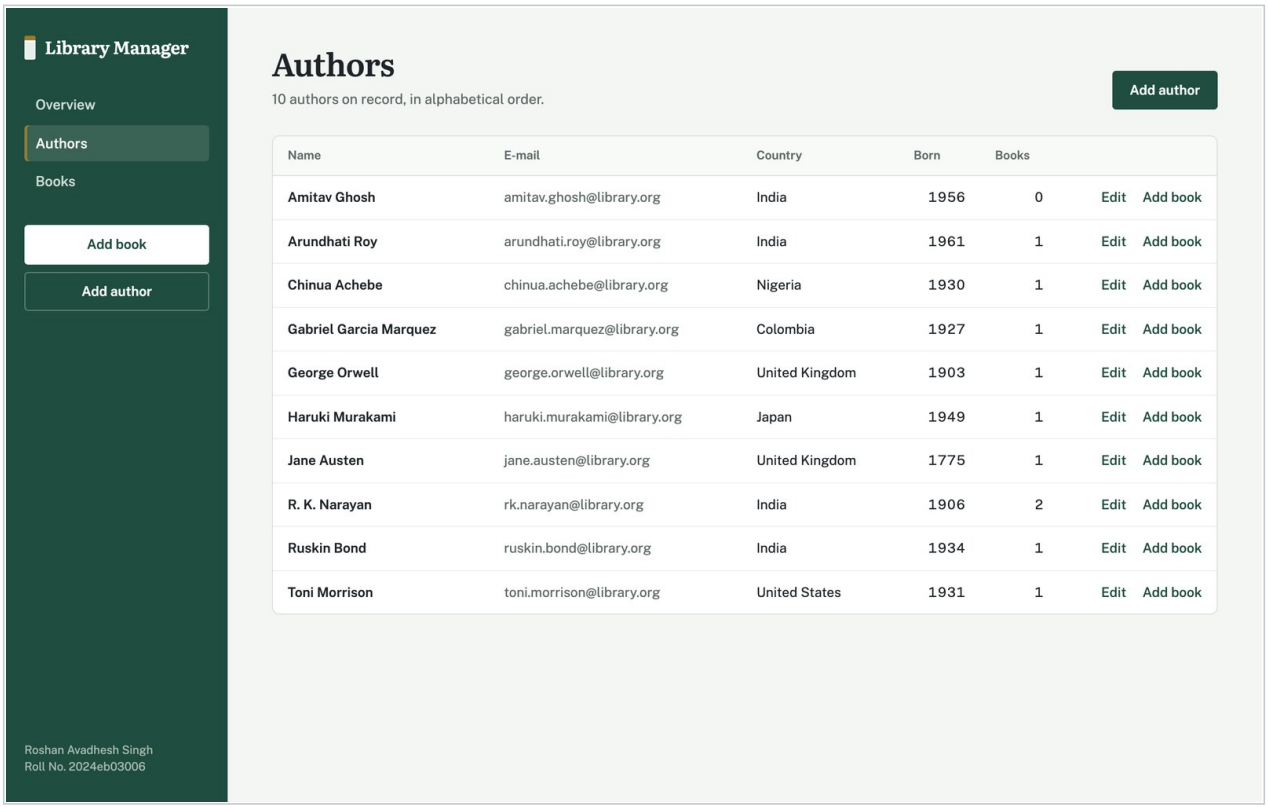


Figure 7. The 10 seeded authors on the Authors page (author count from a LEFT JOIN, so authors without books show 0)

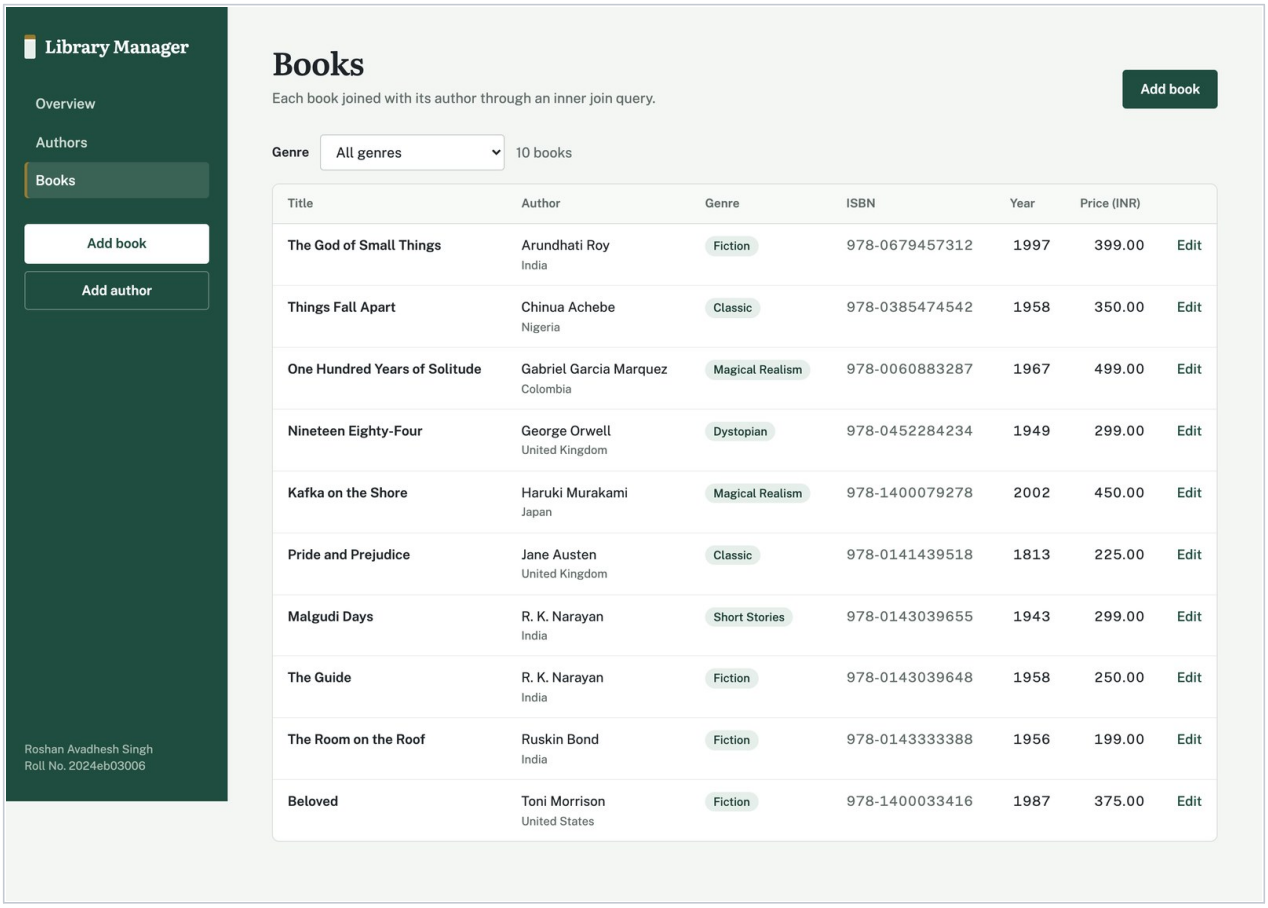


Figure 8. The 10 seeded books, each joined with its author

4. Create operation

Each entity has an add form built with the Spring form tag library. The form is bound to a model attribute (`Author` for authors, and a `BookForm` DTO for books, because the book form carries the selected author's id rather than a whole Author object). Creation happens in two steps: a GET request shows the empty form, and a POST request validates and saves it.

4.1 Controller

```
@GetMapping("/new")
public String showCreateForm(Model model) {
    model.addAttribute("author", new Author());
    return formView(model, false);
}

@PostMapping
public String create(@Valid @ModelAttribute("author") Author author,
                     BindingResult result, Model model, RedirectAttributes redirect) {
    if (result.hasErrors()) {
        return formView(model, false);                 // redisplay with field errors
    }
    try {
        Author saved = authorService.create(author);
        redirect.addFlashAttribute("successMessage",
                "Author \"" + saved.getName() + "\" was added.");
        return "redirect:/authors";                    // Post/Redirect/Get
    } catch (DuplicateResourceException ex) {
        result.rejectValue(ex.getField(), "duplicate", ex.getMessage());
        return formView(model, false);
    }
}
```

Figure 9. AuthorController create methods (BookController follows the same pattern)

4.2 Service

The service normalises the input, checks for an existing e-mail before saving, and calls `saveAndFlush` so that any unique-constraint violation raised by the database happens inside the try block. The low-level `DataIntegrityViolationException` is translated into a `DuplicateResourceException` that names the offending field.

```
@Transactional
public Author create(Author author) {
    normalise(author);                                  // trim, lower-case e-mail
    if (authorRepository.existsByEmailIgnoreCase(author.getEmail())) {
        throw duplicateEmail(author.getEmail());
    }
    author.setId(null);
    return saveOrTranslate(author);
}

private Author saveOrTranslate(Author author) {
    try {
        return authorRepository.saveAndFlush(author);
    } catch (DataIntegrityViolationException ex) {
        throw duplicateEmail(author.getEmail());       // e.g. a concurrent insert
    }
}
```

Figure 10. AuthorService.create with integrity-violation handling

4.3 Screenshots

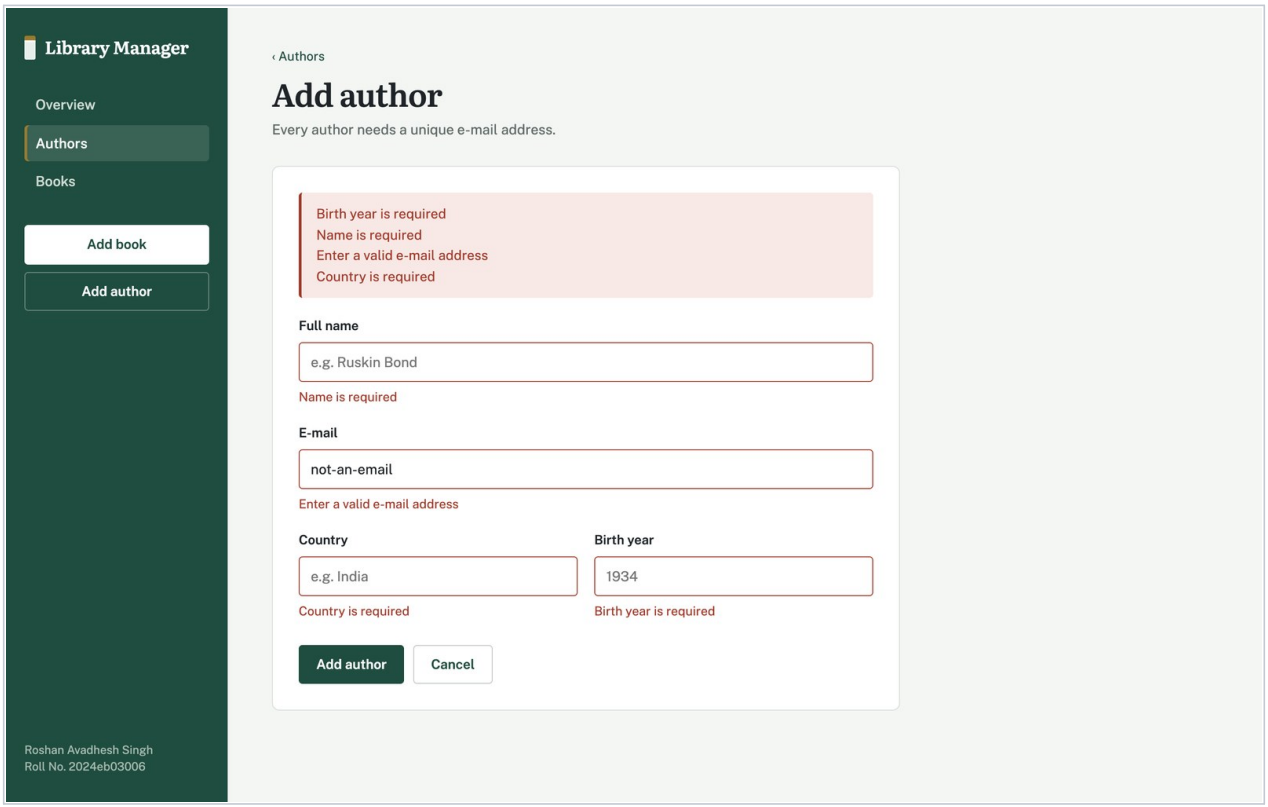


Figure 11. Submitting an incomplete form shows a summary and an error under each invalid field

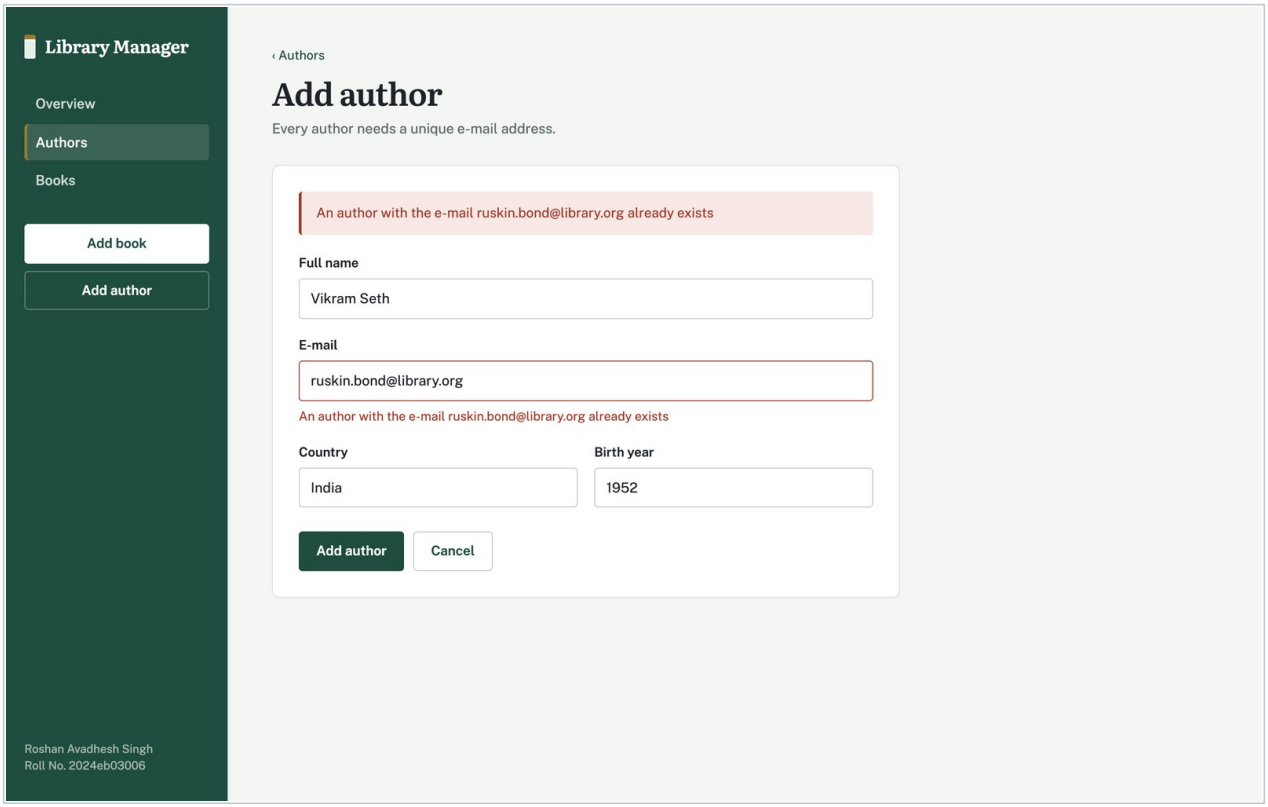


Figure 12. Integrity violation: an author with this e-mail already exists, so nothing is saved

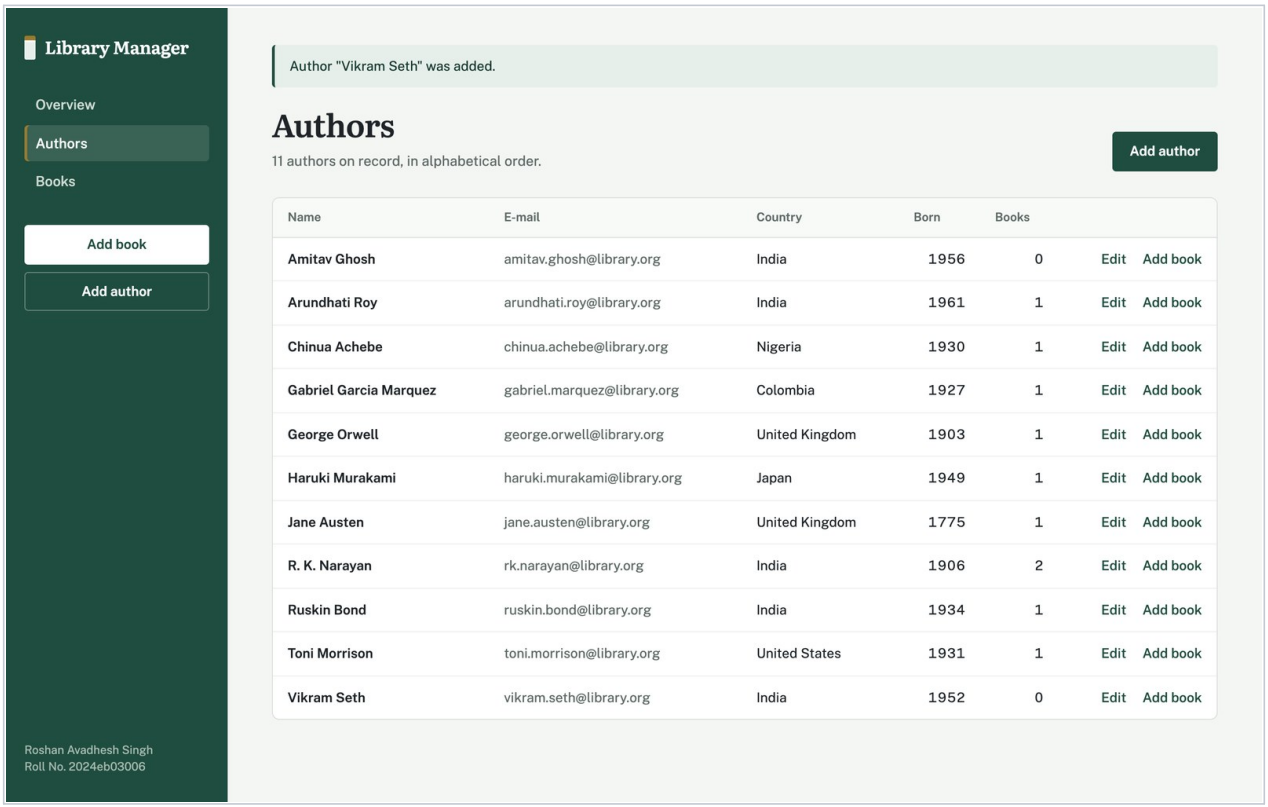


Figure 13. After correcting the e-mail, the author is saved and a confirmation is shown

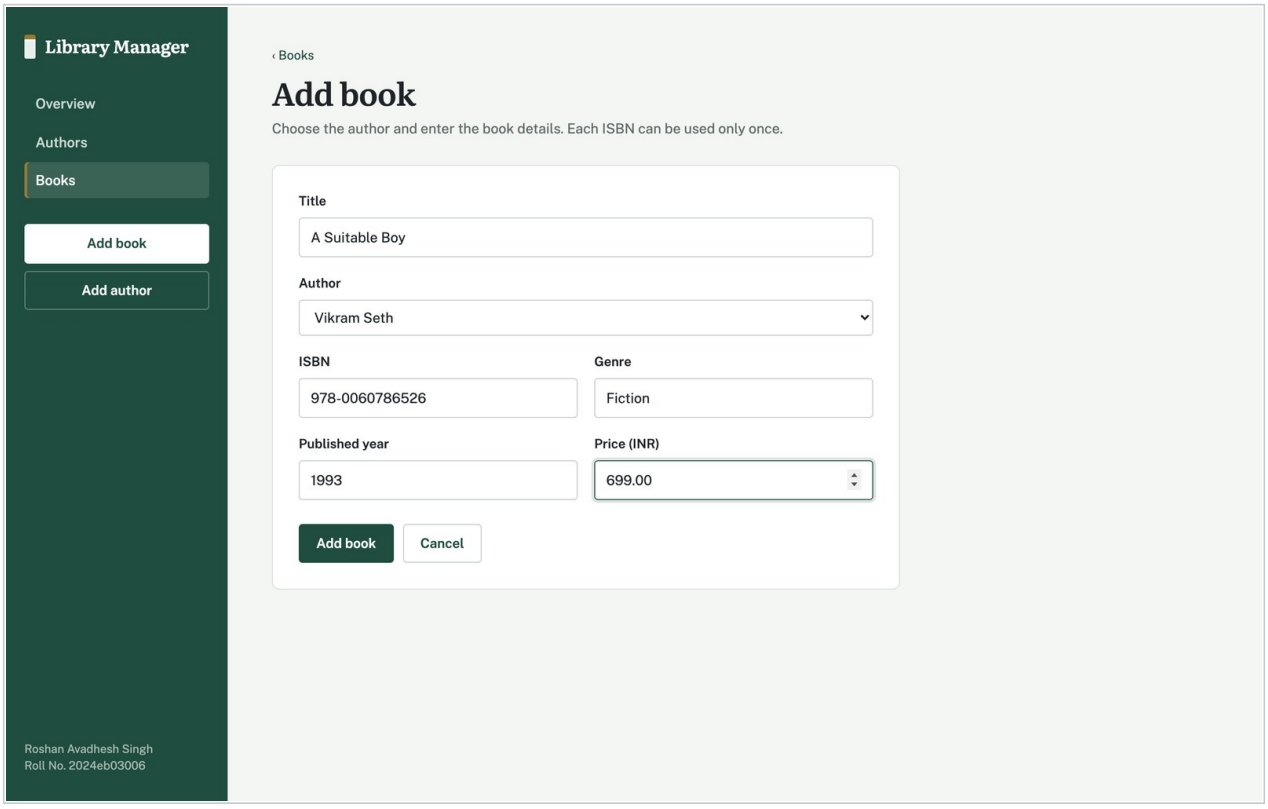


Figure 14. The add book form, with the author chosen from a drop-down list

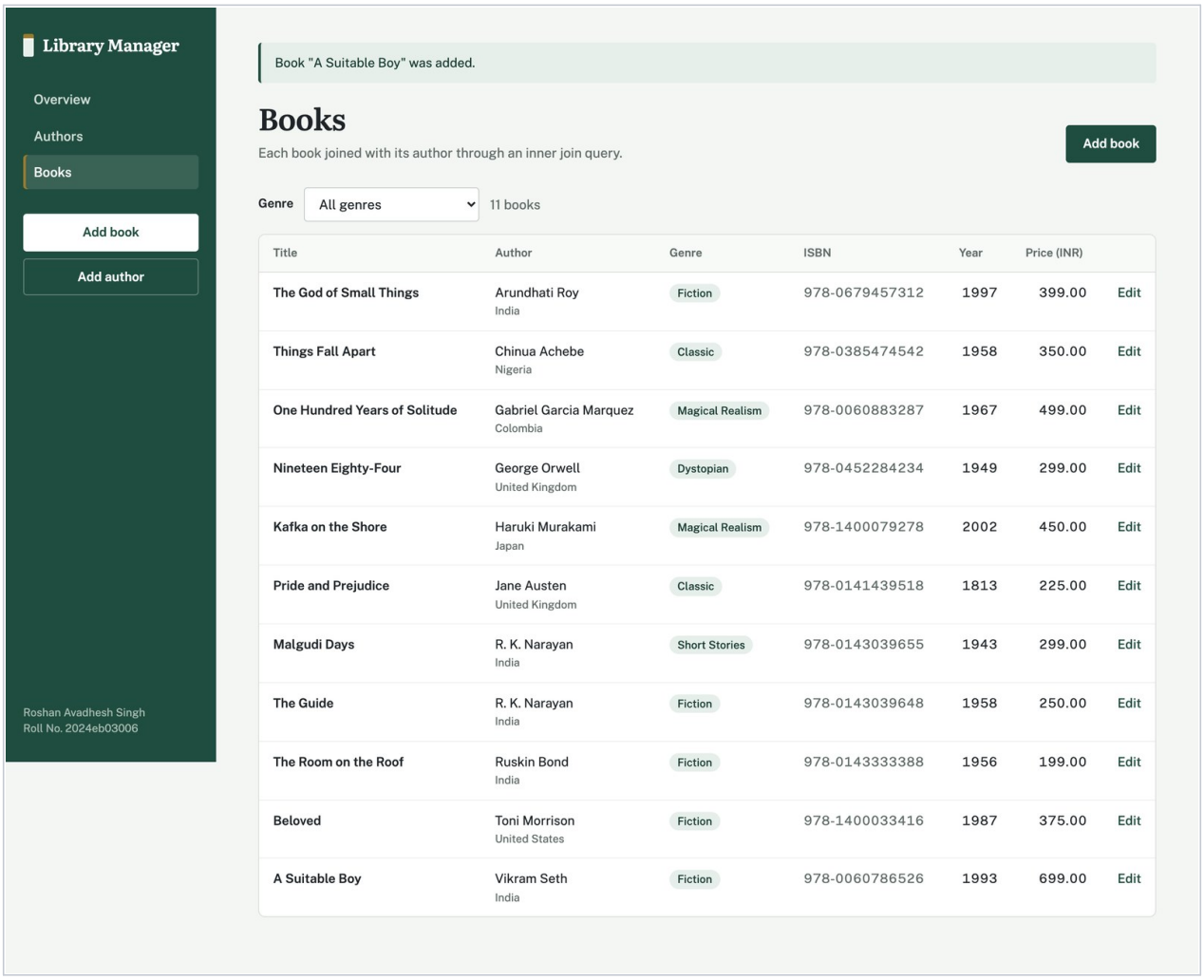


Figure 15. The new book appears in the catalogue with its author

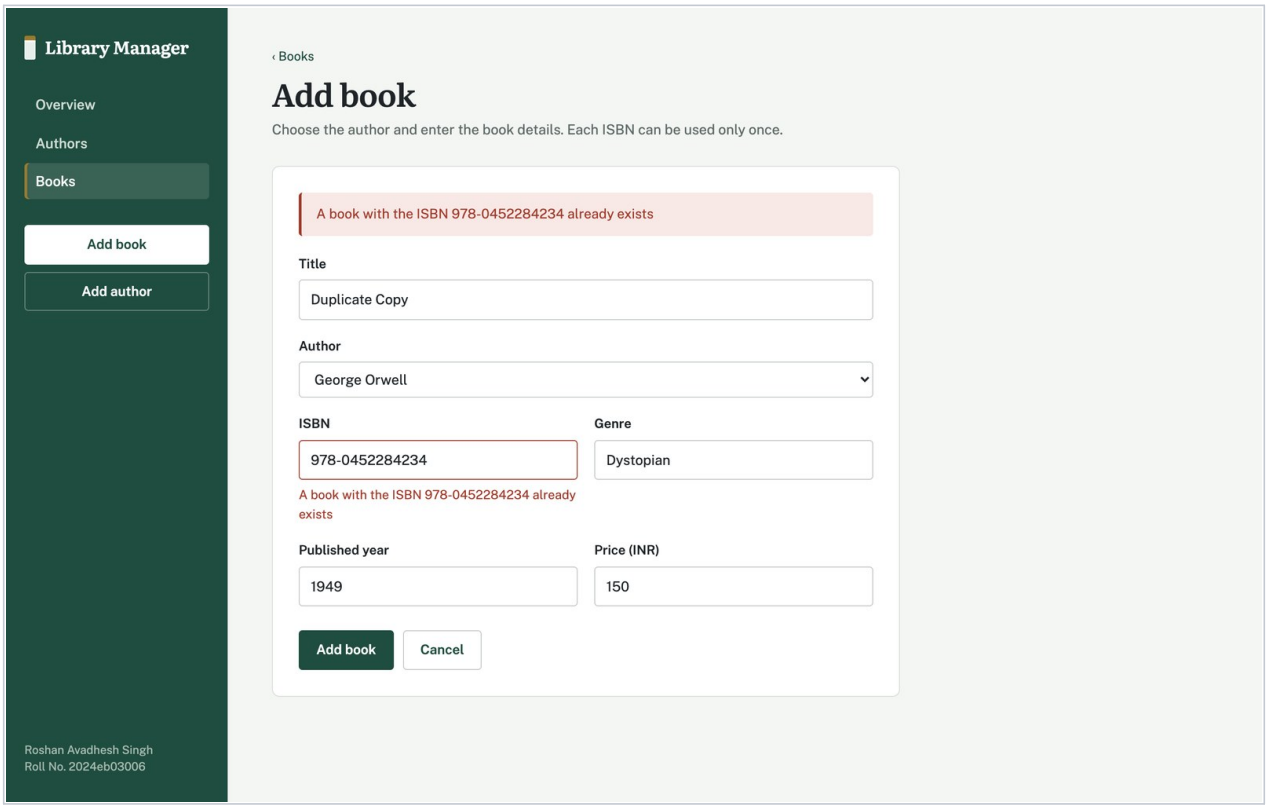


Figure 16. Integrity violation for books: the ISBN is already in use

5. Read operation and custom inner join

The Books page is the main read view. Its data comes from a custom JPQL query in `BookRepository` that performs an **INNER JOIN** between books and authors. A JPQL constructor expression maps each joined row straight into a `BookAuthorView` record, so the view receives exactly the columns it needs from both tables in a single SQL statement. A second version of the query filters the join by genre.

```
@Query("""
       SELECT new com.roshan.library.dto.BookAuthorView(
              b.id, b.title, b.isbn, b.genre, b.publishedYear, b.price,
              a.id, a.name, a.country)
       FROM Book b
       INNER JOIN b.author a
       ORDER BY a.name ASC, b.title ASC
       """)
List<BookAuthorView> findAllBooksWithAuthors();

@Query("""
       SELECT new com.roshan.library.dto.BookAuthorView( ...same columns... )
       FROM Book b
       INNER JOIN b.author a
       WHERE LOWER(b.genre) = LOWER(:genre)
       ORDER BY a.name ASC, b.title ASC
       """)
List<BookAuthorView> findBooksWithAuthorsByGenre(@Param("genre") String genre);
```

Figure 17. Custom inner join queries in BookRepository

```
select b1_0.id, b1_0.title, b1_0.isbn, b1_0.genre, b1_0.published_year, b1_0.price,
       b1_0.author_id, a1_0.name, a1_0.country
from books b1_0
join authors a1_0 on a1_0.id = b1_0.author_id
order by a1_0.name, b1_0.title
```

Figure 18. SQL generated by Hibernate for findAllBooksWithAuthors (captured from the test log)

A second inner join in `AuthorRepository` groups the joined rows to count each author's books. Because it is an inner join, authors with no books are excluded; the overview page uses it to rank authors by the number of books in the collection.

```
@Query("""
       SELECT new com.roshan.library.dto.AuthorSummary(
              a.id, a.name, a.email, a.country, a.birthYear, COUNT(b.id))
       FROM Author a
       INNER JOIN a.books b
       GROUP BY a.id, a.name, a.email, a.country, a.birthYear
       ORDER BY COUNT(b.id) DESC, a.name ASC
       """)
List<AuthorSummary> findAuthorsWithBookCount();
```

Figure 19. Inner join with aggregation in AuthorRepository

5.1 Service, controller and view

```
// BookService
public List<BookAuthorView> findWithAuthorsByGenre(String genre) {
    if (genre == null || genre.isBlank()) return bookRepository.findAllBooksWithAuthors();
    return bookRepository.findBooksWithAuthorsByGenre(genre.trim());
}

// BookController
@GetMapping
public String list(@RequestParam(required = false) String genre, Model model) {
    model.addAttribute("rows", bookService.findWithAuthorsByGenre(genre));
    model.addAttribute("genres", bookService.findGenres());
    model.addAttribute("selectedGenre", genre == null ? "" : genre);
    return "books/list";
}
```

Figure 20. The service and controller pass the join result to the view

```
<c:forEach var="row" items="${rows}">
  <tr>
    <td class="strong">${row.title}</td>
    <td>${row.authorName}<span class="sub">${row.authorCountry}</span></td>
    <td><span class="tag">${row.genre}</span></td>
    <td class="mono">${row.isbn}</td>
    <td class="num">${row.publishedYear}</td>
    <td class="num"><fmt:formatNumber value="${row.price}" minFractionDigits="2"/></td>
    <td class="actions"><a href="<c:url value='/books/${row.bookId}/edit'/>">Edit</a></td>
  </tr>
</c:forEach>
```

Figure 21. books/list.jsp binds each joined row with JSTL and Expression Language

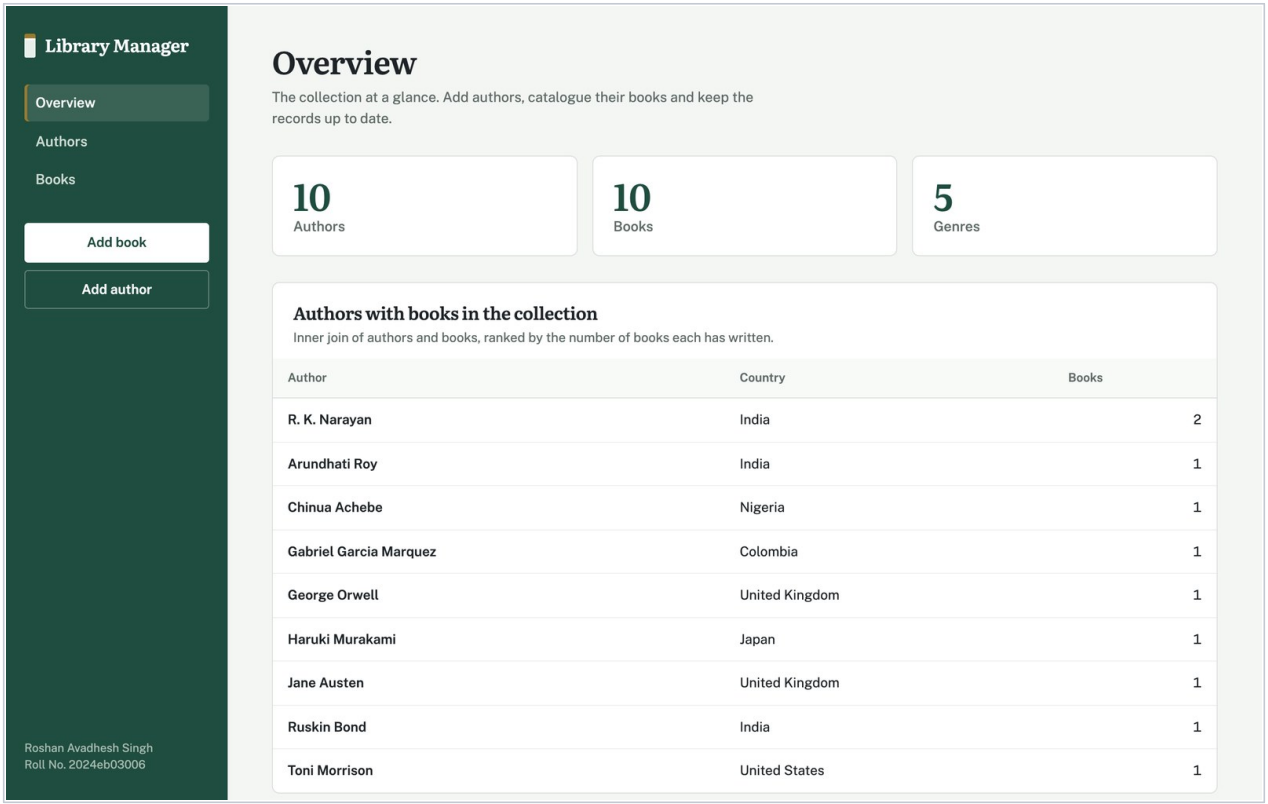


Figure 22. Overview page: totals and the inner join of authors with their book counts

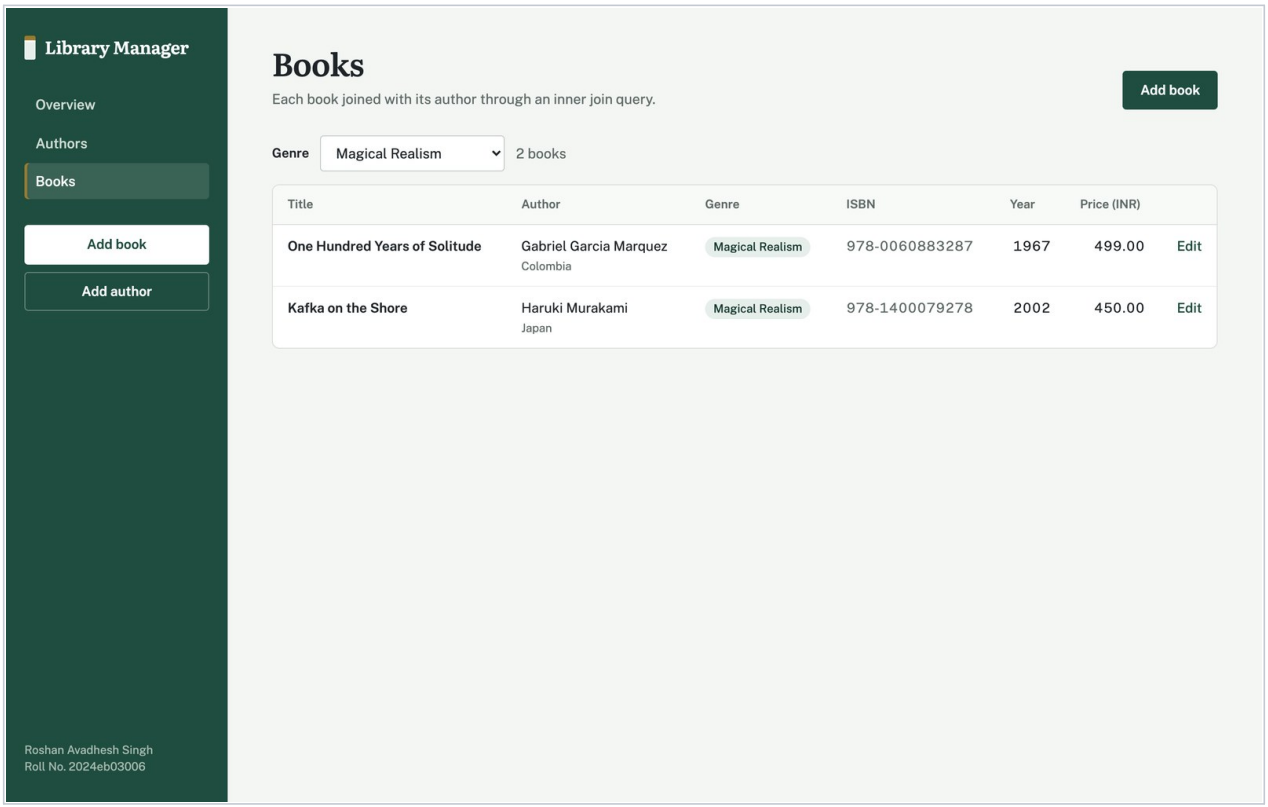


Figure 23. The inner join filtered by genre (Magical Realism)

6. Update operation

Every row in the author and book lists has an Edit link. The GET handler loads the existing record and pre-fills the same JSP form used for creation (an `editing` flag changes the heading, the form action and the button label). The POST handler validates the input and passes it to the service, which copies the changes onto the managed entity and saves it. When updating, the uniqueness check excludes the record being edited, so saving a record without changing its e-mail or ISBN is allowed.

```
// BookController
@GetMapping("/{id}/edit")
public String showEditForm(@PathVariable Long id, Model model) {
    model.addAttribute("book", BookForm.fromEntity(bookService.findById(id)));
    return formView(model, true);
}

@PostMapping("/{id}")
public String update(@PathVariable Long id, @Valid @ModelAttribute("book") BookForm form,
                     BindingResult result, Model model, RedirectAttributes redirect) {
    form.setId(id);
    if (result.hasErrors()) return formView(model, true);
    try {
        Book saved = bookService.update(id, form.toEntity(), form.getAuthorId());
        redirect.addFlashAttribute("successMessage",
                "Book \"" + saved.getTitle() + "\" was updated.");
        return "redirect:/books";
    } catch (DuplicateResourceException ex) {
        result.rejectValue(ex.getField(), "duplicate", ex.getMessage());
        return formView(model, true);
    }
}

// BookService
@Transactional
public Book update(Long id, Book changes, Long authorId) {
    Book existing = findById(id);                        // JOIN FETCH with author
    if (bookRepository.existsByIsbnAndIdNot(changes.getIsbn(), id)) {
        throw duplicateIsbn(changes.getIsbn());
    }
    existing.setTitle(changes.getTitle());
    existing.setIsbn(changes.getIsbn());
    existing.setGenre(changes.getGenre());
    existing.setPublishedYear(changes.getPublishedYear());
    existing.setPrice(changes.getPrice());
    existing.setAuthor(loadAuthor(authorId));          // the author can be changed too
    return saveOrTranslate(existing);
}
```

Figure 24. Update handling in BookController and BookService

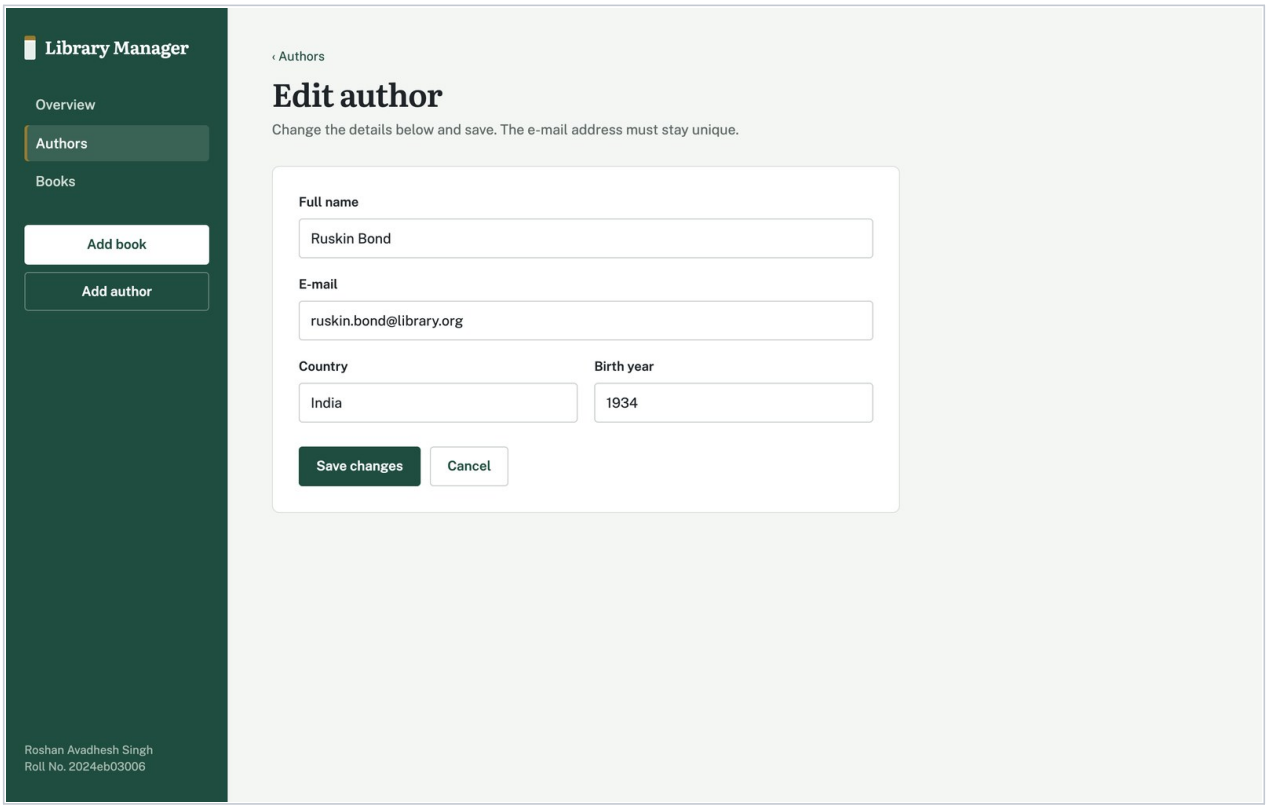


Figure 25. Editing an author: the form is pre-filled with the current values

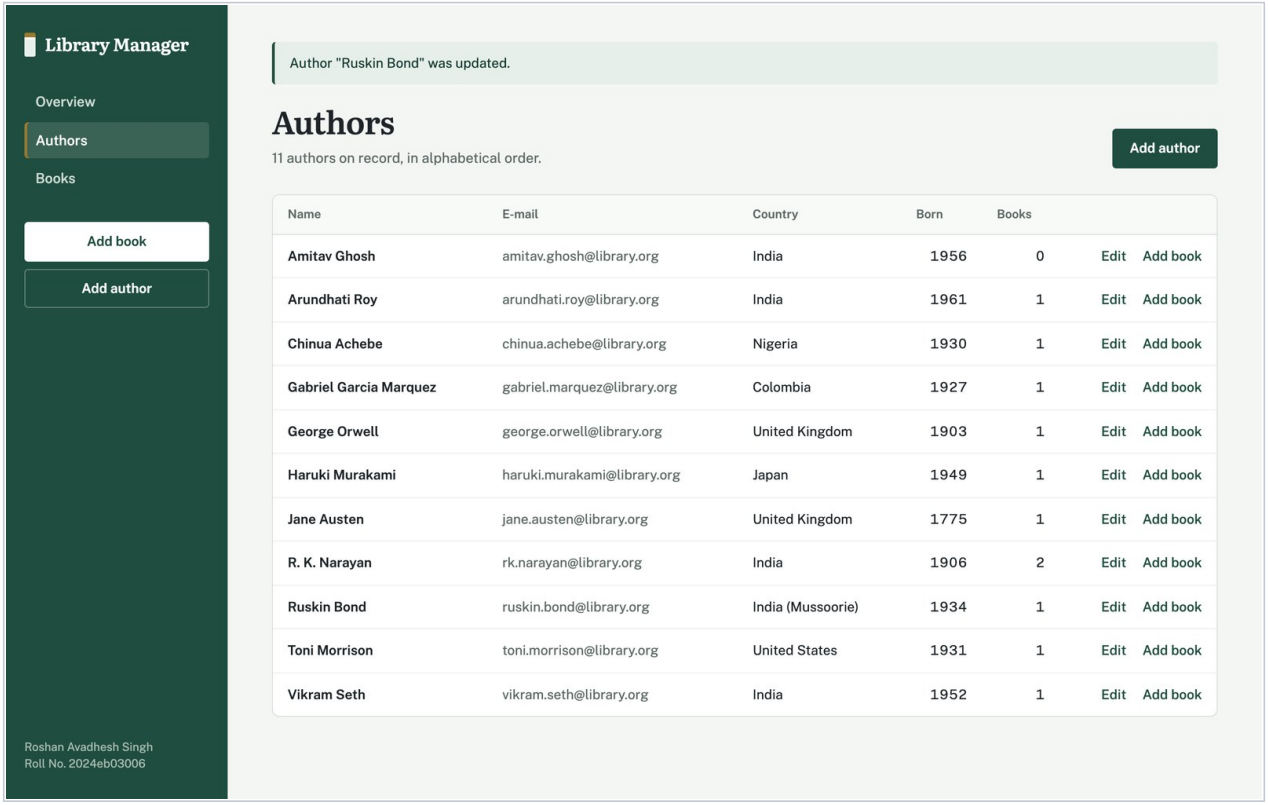


Figure 26. The updated country is saved and shown in the list

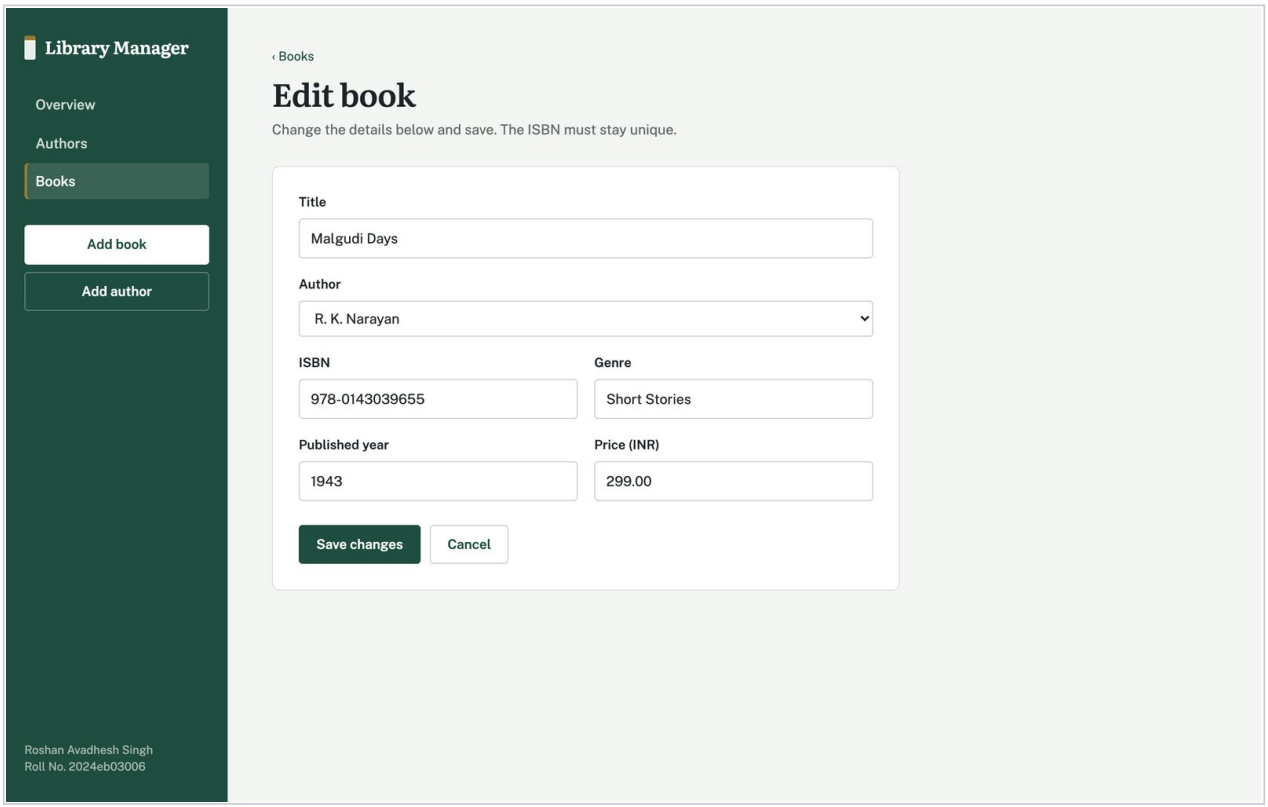


Figure 27. Editing a book (Malgudi Days)

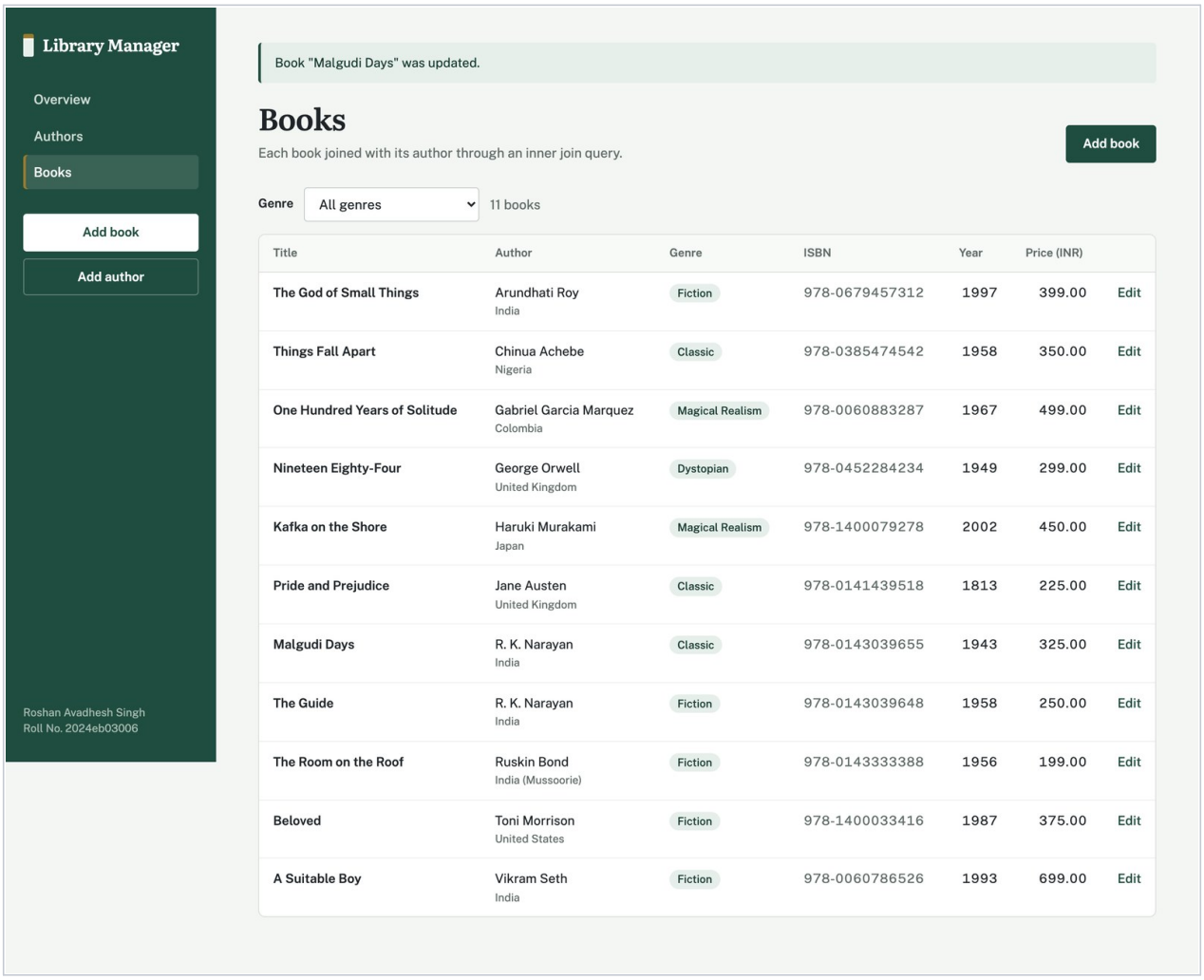


Figure 28. The book's new genre and price after the update

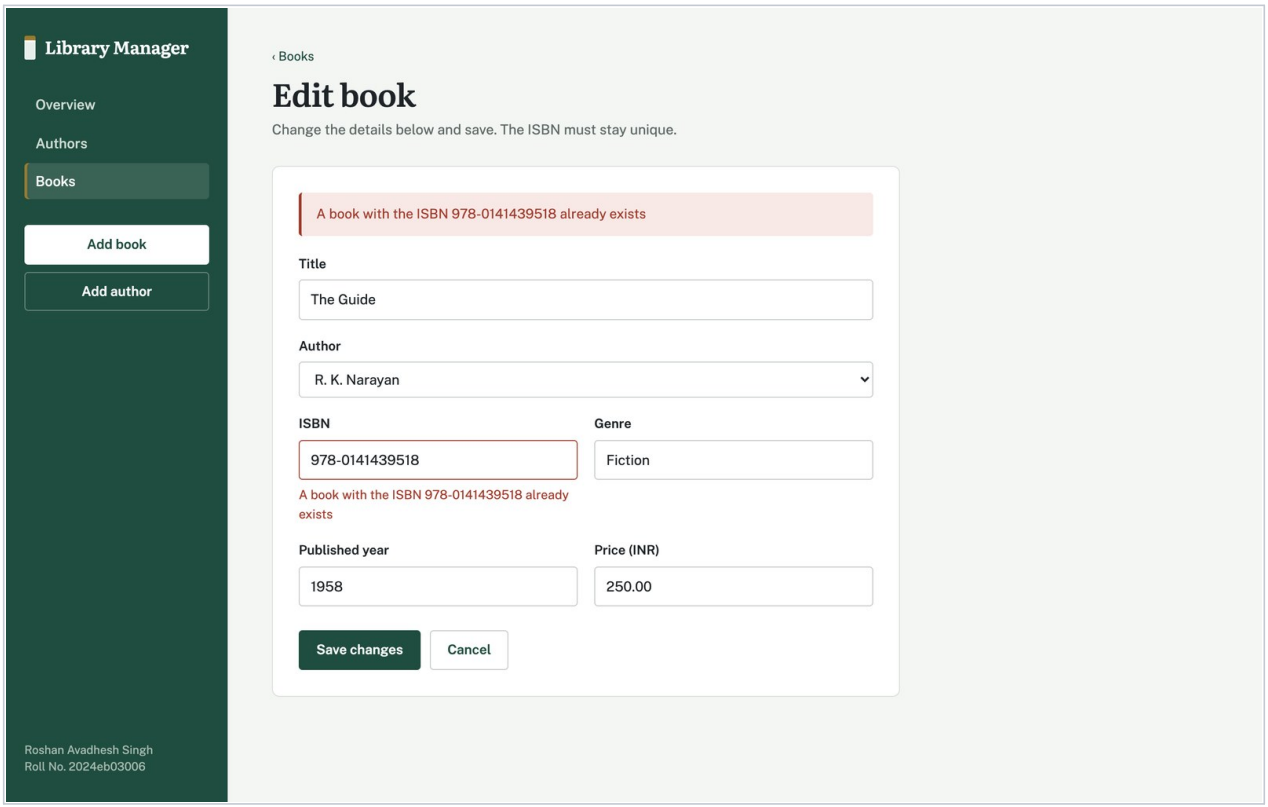


Figure 29. Integrity violation on update: the new ISBN belongs to another book

7. Exception handling

Errors are handled at three levels, so that the user always gets a clear message and the database is never left in an inconsistent state:

- **Validation errors** are caught by Bean Validation (`@Valid` with `BindingResult`) and shown next to each field.
- **Integrity violations** (duplicate e-mail or ISBN) are detected by the service, either by the pre-check or by catching `DataIntegrityViolationException` from `saveAndFlush`. They are rethrown as `DuplicateResourceException` and the controller attaches the message to the right form field. Because the service method is transactional, the failed write is rolled back.
- **Missing records** raise `ResourceNotFoundException`, which a `@ControllerAdvice` turns into a 404 page. The same advice catches any integrity violation that escapes a controller and returns a 409 page.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ModelAndView handleNotFound(ResourceNotFoundException ex) {
        return errorPage("Record not found", ex.getMessage());
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ModelAndView handleIntegrity(DataIntegrityViolationException ex) {
        return errorPage("Data integrity violation",
                "The change could not be saved because it conflicts with existing data.");
    }
}
```

Figure 30. Application-wide exception handling

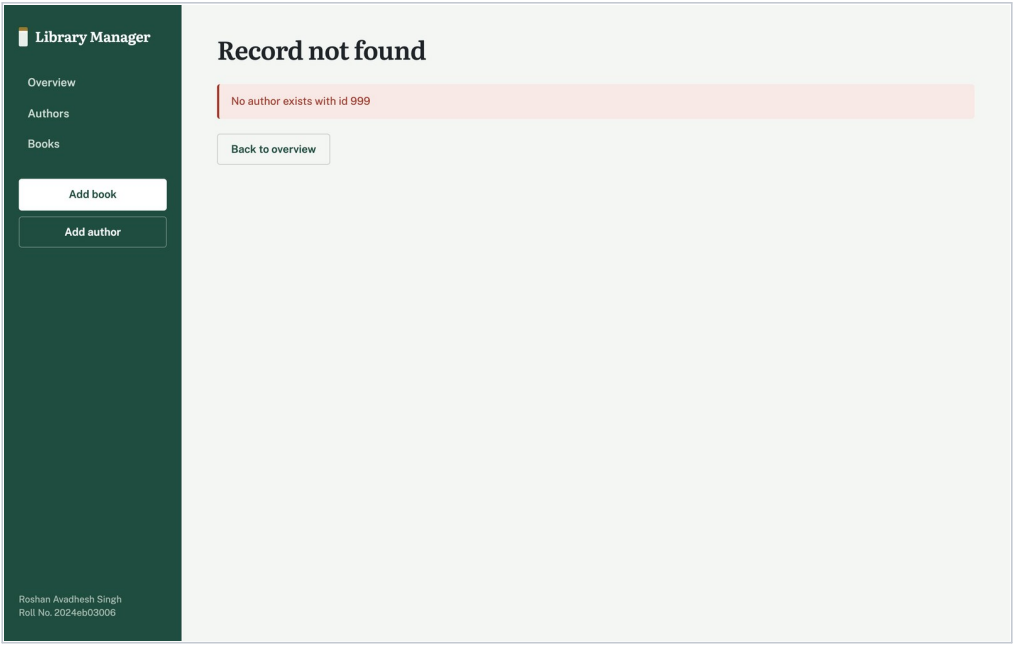


Figure 31. Requesting an author that does not exist returns a 404 page

8. User interface

The views share a header and footer through JSP static includes (`header.jspf`, `footer.jspf`), which provide the navigation sidebar and the success message area. A single stylesheet, `app.css`, defines the visual design:

- A dark green sidebar with the navigation, quick actions for adding a book or an author, and the student's name and roll number.
- Tables with right-aligned, tabular numbers, genre tags and an Edit action on every row.
- Forms with clear labels, placeholders showing the expected format, red highlighting and messages for invalid fields, and a genre suggestion list.
- Visible keyboard focus, and a responsive layout that stacks the sidebar above the content on narrow screens, where wide tables scroll inside their panel.

9. Testing

The repository layer is tested with `@DataJpaTest`, which starts only JPA against an in-memory H2 database and rolls back after each test. The service layer is tested in isolation with JUnit 5 and Mockito, with the repositories replaced by mocks. All **25 tests pass** (`mvn test`).

Test class	Tests	What is verified
BookRepositoryTest (@DataJpaTest)	10	Inner join returns every book with its author in the right order; genre filter is case-insensitive; author inner join excludes authors without books; left join includes them with a count of 0; JOIN FETCH loads the author; existsByIsbn checks; distinct genres; unique ISBN and e-mail constraints raise DataIntegrityViolationException; a book without an author is rejected.
AuthorServiceTest (Mockito)	6	Create trims and lower-cases input and saves; duplicate e-mail is rejected before saving; database integrity violation is translated; update copies all fields; update rejects an e-mail used by another author; missing id throws.
BookServiceTest (Mockito)	9	Create links the author; duplicate ISBN is rejected; unknown author throws; database integrity violation is translated; update changes fields and author; update rejects another book's ISBN; missing book throws; a blank genre uses the full join and a genre uses the filtered join.

```
@Test
void innerJoinOnAuthorsExcludesAuthorsWithoutBooks() {
    List<AuthorSummary> summaries = authorRepository.findAuthorsWithBookCount();

    assertThat(summaries).extracting(AuthorSummary::name)
            .containsExactly("George Orwell", "Jane Austen");      // "New Writer" has no books
    assertThat(summaries.get(0).bookCount()).isEqualTo(2L);
}

@Test
void duplicateIsbnViolatesUniqueConstraint() {
    Book copy = book("Another Title", "978-0452284234", "Dystopian", orwell);
    assertThatThrownBy(() -> bookRepository.saveAndFlush(copy))
            .isInstanceOf(DataIntegrityViolationException.class);
}
```

Figure 32. Examples of repository tests

```
@Test
void createRejectsDuplicateEmailBeforeSaving() {
    Author input = new Author("Copy", "taken@library.org", "India", 1950);
    when(authorRepository.existsByEmailIgnoreCase("taken@library.org")).thenReturn(true);

    assertThatThrownBy(() -> authorService.create(input))
            .isInstanceOf(DuplicateResourceException.class)
            .extracting("field").isEqualTo("email");
    verify(authorRepository, never()).saveAndFlush(any());
}
```

Figure 33. Example of a Mockito service test

```
BookRepositoryTest   Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
AuthorServiceTest    Tests run: 6,  Failures: 0, Errors: 0, Skipped: 0
BookServiceTest      Tests run: 9,  Failures: 0, Errors: 0, Skipped: 0
Total                Tests run: 25, Failures: 0, Errors: 0, Skipped: 0   BUILD SUCCESS
```

Figure 34. Surefire test results

In addition to the unit tests, every create, read and update operation was exercised through the web interface, as shown in the screenshots in sections 3 to 7.

10. Challenges faced and solutions

Challenge	How it was overcome
JSP pages not found when running as a JAR	Spring Boot does not serve `src/main/webapp` from an executable JAR. I placed the JSPs under `src/main/resources/META-INF/resources/WEB-INF/jsp`, added `tomcat-embed-jasper` and the Jakarta JSTL dependencies, and set the view prefix and suffix.
Integrity violations surfacing too late	With a plain `save`, Hibernate delays the INSERT until the transaction commits, so a duplicate key error escaped the service as a generic exception. Using `saveAndFlush` forces the statement to run immediately, inside a try block that translates it into a field-specific `DuplicateResourceException`. A pre-check with `existsBy...` gives a fast, friendly message in the common case.
Allowing an unchanged unique value on update	A plain `existsByIsbn` check rejected every update, because the book's own ISBN already exists. I added `existsByIsbnAndIdNot` and `existsByEmailIgnoreCaseAndIdNot`, which ignore the record being edited.
LazyInitializationException on the edit page	The book's author is loaded lazily and Open Session in View is disabled, so reading the author after the service returned failed. A dedicated `INNER JOIN FETCH` query loads the book and its author together. For the list page, the projection query returns plain values, so no lazy loading is needed at all.
Binding the selected author in the book form	Binding a nested `author.id` path directly onto the entity created half-filled Author objects. A separate `BookForm` DTO with an `authorId` field keeps form binding simple and lets the service load the real author.
Reading Java records in JSP	Expression Language looks for JavaBean getters, which Java records do not generate. I added `getX()` accessors to the projection records so that `${row.title}` works in the views.
Layout breaking on narrow screens	On small screens the single-column grid grew to fit the widest table, which stretched the sidebar buttons. Using `minmax(0, 1fr)` for the grid column and letting tables scroll within their panel fixed it.

11. Running the project

The source code is available at https://github.com/roshanasingh4/library-manager-springboot.

Step	Command or address
Clone	`git clone https://github.com/roshanasingh4/library-manager-springboot.git`
Run the tests	`mvn test`
Start the application	`mvn spring-boot:run`
Open the application	`http://localhost:8080`
Inspect the database	`http://localhost:8080/h2-console` (JDBC URL `jdbc:h2:mem:librarydb`, user `sa`)

Requirements: JDK 21 and Maven 3.9 or later. No external database is needed.